HyperionDev

Transformers and Large Language Models(LLMs)

Model Answer Approach

Visit our website

Auto-graded task 1

Although this solution appears to be a straightforward collection of loops and arrays, it demonstrates the core mathematical operations that power modern Large Language Models. Using plain JavaScript rather than AI libraries helps reveal how language models rely on fundamental arithmetic to process and understand text.

The script works with a dataset called `textSequence` containing the words *"React"*, *"Manages"*, and *"State"*. Each word is represented by Query, Key, and Value vectors rather than letters or text. The code focuses on the word **`"State"`** and evaluates how strongly it relates to the other words in the sequence by comparing its Query vector against each token's Key vector.

To calculate these relationships, the script uses the `computeDotProduct()` function. This function multiplies matching values from two vectors and sums the results to produce a relevance score. Higher scores indicate a stronger contextual relationship between words. For example, the Query vector for *"State"* produces a much higher score when compared with its own Key vector than when compared with the Key vector for *"Manages"*, demonstrating how transformers identify meaningful contextual connections.

To maintain mathematical stability, the script applies a scaling step by dividing each raw attention score by 2. This mimics the normalization process used in real transformer architectures, where scaling helps prevent attention values from becoming excessively large as models process increasingly complex inputs.

Rather than modifying the original dataset, the solution uses JavaScript's `.map()` method and the object spread operator (**`...`**) to create a new array containing the calculated `rawScore` and `scaledScore` values. This preserves the original data structure while generating a clean output that can be easily validated by an automated grading system.

Together, these operations simulate the core workflow of a transformer attention mechanism: representing language as vectors, measuring relationships through dot products, stabilizing calculations through scaling, and producing context-aware outputs.